

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 1: 1. Модели данных и состояния игры (Пишем в конце файла ViewModel)

```
// Состояния игры
enum class GameState {
    NOT_STARTED, PLAYING, GAME_OVER
}

// Модель трубы
data class Pipe(
    val x: Float,
    val gapY: Float,
    val width: Float = 160f,
    val gapHeight: Float = 330f,
    val passed: Boolean = false
)

// Общее состояние экрана
data class FlappyState(
    val gameState: GameState = GameState.NOT_STARTED,
    val birdY: Float = 800f,
    val pipes: List<Pipe> = emptyList(),
    val score: Int = 0,
    val highScore: Int = 0
)
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 2: 2. ViewModel: Константы и настройки физики

```
class FlappyBirdViewModel : ViewModel() {
    companion object {
        // Координаты игрового поля
        const val BOARD_WIDTH = 1000f
        const val BOARD_HEIGHT = 1600f
        const val GROUND_Y = 1450f

        // Настройки птички и физики - сюда потом по заданию изменения
        const val BIRD_X = 250f
        const val BIRD_RADIUS = 35f
        private const val FRAME_RATE_MS = 16L
        private const val GRAVITY = 0.7f
        private const val JUMP_VELOCITY = -15f
        private const val TERMINAL_VELOCITY = 18f
        private const val PIPE_SPEED = 7.5f
        private const val MIN_GAP_Y = 350f
        private const val MAX_GAP_Y = 1100f
    }

    private val _state = MutableStateFlow(FlappyState())
    val state: StateFlow<FlappyState> = _state.asStateFlow()
    private var gameJob: Job? = null
    private var birdVelocity = 0f
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 3: 3. ViewModel: Управление прыжком и запуском - всё в теле ViewModel

```
fun onTap() {
    when (_state.value.gameState) {
        GameState.NOT_STARTED -> startGame()
        GameState.PLAYING -> jump()
        GameState.GAME_OVER -> resetGame()
    }
}

private fun startGame() {
    _state.value = FlappyState(
        gameState = GameState.PLAYING,
        highScore = _state.value.highScore
    )
    birdVelocity = JUMP_VELOCITY
    startGameLoop()
}

private fun jump() {
    birdVelocity = JUMP_VELOCITY
}

private fun resetGame() {
    startGame()
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 4: 4. ViewModel: Главный игровой цикл (Game Loop)

```
private fun startGameLoop() {
    gameJob?.cancel()
    gameJob = viewModelScope.launch {
        while (isActive &&
            _state.value.gameState == GameState.PLAYING) {
            updateGame()
            delay(FRAME_RATE_MS)
        }
    }
}

private fun endGame() {
    gameJob?.cancel()
    val finalScore = _state.value.score
    val currentHighScore = _state.value.highScore
    _state.value = _state.value.copy(
        gameState = GameState.GAME_OVER,
        highScore = maxOf(finalScore, currentHighScore)
    )
}

override fun onCleared() {
    super.onCleared()
    gameJob?.cancel()
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 5: 5. ViewModel: Логика движения птички и труб

```
private fun updateGame() {
    val currentState = _state.value
    // 1. Физика падения
    birdVelocity = (birdVelocity + GRAVITY)
        .coerceAtMost(TERMINAL_VELOCITY)
    val newBirdY = currentState.birdY + birdVelocity

    // 2. Движение труб
    val updatedPipes = currentState.pipes
        .map { it.copy(x = it.x - PIPE_SPEED) }
        .filter { it.x + it.width > 0 }
        .toMutableList()

    // Создание новой трубы
    val lastPipe = updatedPipes.lastOrNull()
    if (lastPipe == null || lastPipe.x < BOARD_WIDTH - 450f) {
        val randomGapY = Random.nextFloat() *
            (MAX_GAP_Y - MIN_GAP_Y) + MIN_GAP_Y
        updatedPipes.add(Pipe(x = BOARD_WIDTH, gapY = randomGapY))
    }

    checkCollisions(newBirdY, updatedPipes, currentState)
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 6: 6. ViewModel: Проверка столкновений

```
private fun checkCollisions(newBirdY: Float,
    updatedPipes: MutableList<Pipe>, currentState: FlappyState) {
    var collision = false
    var newScore = currentState.score
    val birdLeft = BIRD_X - BIRD_RADIUS
    val birdRight = BIRD_X + BIRD_RADIUS
    val birdTop = newBirdY - BIRD_RADIUS
    val birdBottom = newBirdY + BIRD_RADIUS

    // Столкновение с землей или потолком
    if (birdBottom >= GROUND_Y || birdTop <= 0) collision = true

    // Столкновение с трубами
    for (i in updatedPipes.indices) {
        val pipe = updatedPipes[i]
        if (birdRight > pipe.x && birdLeft < pipe.x + pipe.width) {
            if (birdTop < pipe.gapY - pipe.gapHeight / 2 ||
                birdBottom > pipe.gapY + pipe.gapHeight / 2) {
                collision = true
                break
            }
        }
        if (!pipe.passed && pipe.x + pipe.width / 2 < BIRD_X) {
            updatedPipes[i] = pipe.copy(passed = true)
            newScore++
        }
    }

    if (collision) endGame() else {
        _state.value = currentState.copy(
            birdY = newBirdY, pipes = updatedPipes, score = newScore
        )
    }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 7: 7. Экран игры: Фон и масштабирование Canvas

```
@Composable
fun FlappyBirdScreen(
    birdPainter: Painter? = null,
    pipePainter: Painter? = null,
    viewModel: FlappyBirdViewModel = viewModel()
) {
    val state by viewModel.state.collectAsState()
    Box(modifier = Modifier.fillMaxSize()
        .background(Color(0xFF70C5CF))
        .pointerInput(Unit) {
            detectTapGestures { viewModel.onTap() }
        }
    ) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            val scaleX = size.width / FlappyBirdViewModel.BOARD_WIDTH
            val scaleY = size.height / FlappyBirdViewModel.BOARD_HEIGHT
            withTransform({
                scale(scaleX, scaleY, pivot = Offset(0f, 0f))
            }) {
                // Здесь будет отрисовка (Шаги 8-9)
            }
        }
        // Здесь будет UI (Шаг 10)
    }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 8: 8. Отрисовка: Трубы

```
// Внутри блока Canvas -> withTransform:
state.pipes.forEach { pipe ->
    val gapTop = pipe.gapY - pipe.gapHeight / 2
    val gapBottom = pipe.gapY + pipe.gapHeight / 2
    if (pipePainter != null) {
        // Рисуем картинку трубы
        translate(left = pipe.x, top = 0f) {
            with(pipePainter) { draw(size = Size(pipe.width, gapTop)) }
        }
        translate(left = pipe.x, top = gapBottom) {
            with(pipePainter) {
                draw(size = Size(pipe.width, GROUND_Y - gapBottom))
            }
        }
    } else {
        // Если нет картинки рисуем зеленые прямоугольники
        drawRect(Color(0xFF73C73E), Offset(pipe.x, 0f),
            Size(pipe.width, gapTop))
        drawRect(Color(0xFF73C73E), Offset(pipe.x, gapBottom),
            Size(pipe.width, GROUND_Y - gapBottom))
    }
}
```


СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 9: 9. Отрисовка: Птичка и земля

```
// 1. Рисуем землю
drawRect(Color(0xFFD2B48C), Offset(0f, GROUND_Y),
    Size(BOARD_WIDTH, BOARD_HEIGHT - GROUND_Y))
drawRect(Color(0xFF55A630), Offset(0f, GROUND_Y),
    Size(BOARD_WIDTH, 40f))

// 2. Рисуем птичку
if (birdPainter != null) {
    translate(BIRD_X - BIRD_RADIUS, state.birdY - BIRD_RADIUS) {
        with(birdPainter) {
            draw(size = Size(BIRD_RADIUS * 2, BIRD_RADIUS * 2))
        }
    }
} else {
    drawCircle(Color(0xFFFF9C80E), BIRD_RADIUS,
        Offset(BIRD_X, state.birdY))
    // Глазик
    drawCircle(Color.Black, 4f,
        Offset(BIRD_X + 12f, state.birdY - 10f))
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 10: 10. Интерфейс: Счет и экраны состояния

```
// Внутри Box, после Canvas:
Column(modifier = Modifier.fillMaxSize().padding(32.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.SpaceBetween
) {
    Text("Score: ${state.score}", fontSize = 36.sp,
        fontWeight = FontWeight.Bold, color = Color.White)

    if (state.gameState == GameState.NOT_STARTED) {
        Text("TAP TO START", fontSize = 24.sp, color = Color.White)
    }

    if (state.gameState == GameState.GAME_OVER) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text("GAME OVER", fontSize = 44.sp, color = Color.Red)
            Text("High Score: ${state.highScore}", color = Color.White)
            Spacer(modifier = Modifier.height(16.dp))
            Text("TAP TO RESTART", color = Color.White)
        }
    }
}
```

ЭТАП 11: 11. Использование картинок - 1 птица, 2 трубы

```
Surface(    modifier = Modifier.fillMaxSize(),    color =
MaterialTheme.colorScheme.background) {
    FlappyBirdScreen(painterResource(R.drawable.me),
    painterResource(R.drawable.lucy_promo))}
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Настрой сложность: измени гравитацию (GRAVITY) и скорость труб (PIPE_SPEED). Попробуй сделать так, чтобы проход между трубами (gapHeight) становился меньше, когда игрок набирает больше 10 очков!